Practice NEXT.JS

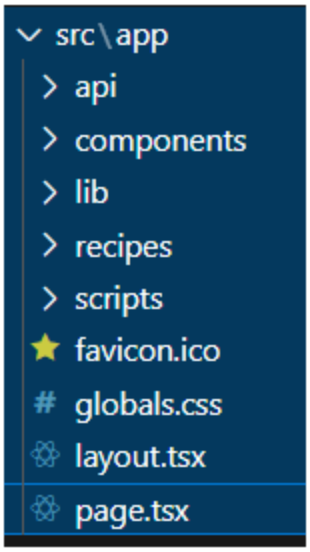
Cooking Recipes Management

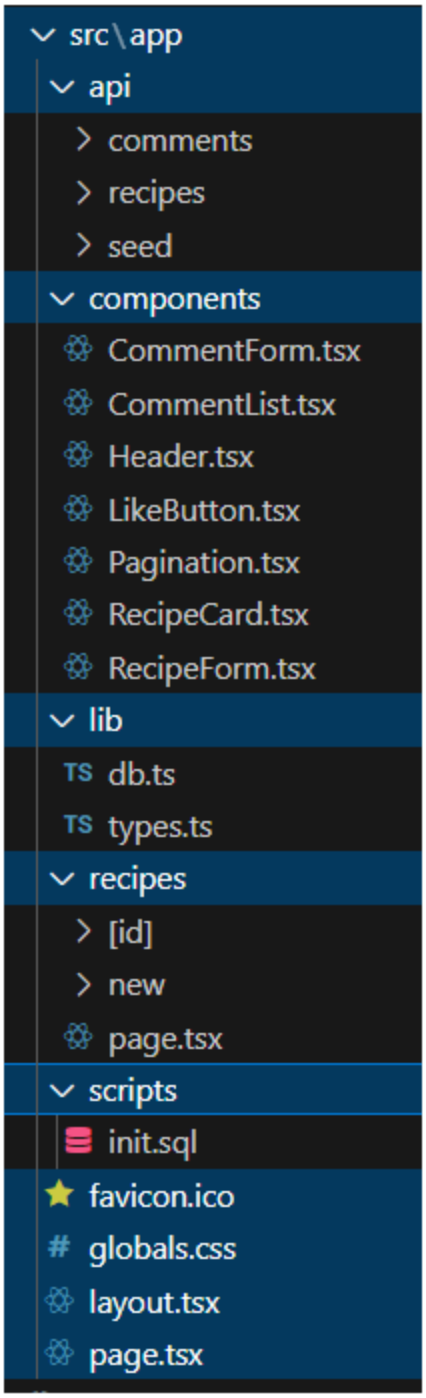
0) Mô tả dự án:

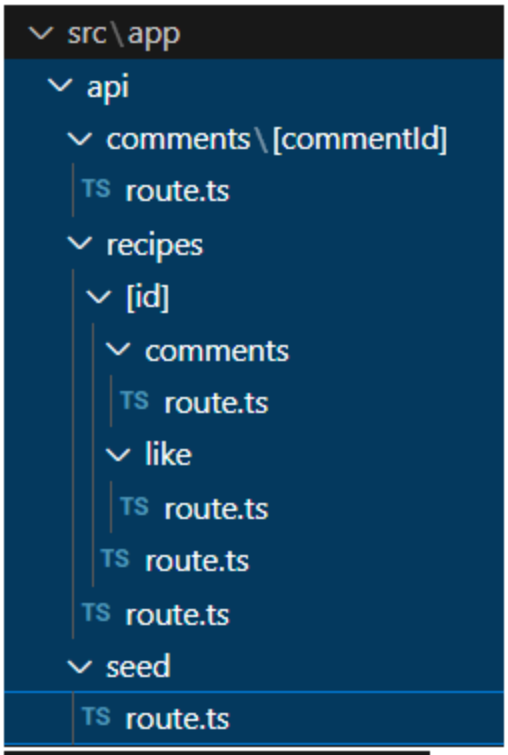
- Tạo project Next.js (App Router).
- Trang web có thể tạo mới Recipes và cho phép Like + Comments.
- Dùng **Database Connection String Neon** sau đó khai báo vào "Biến môi trường" **.env**.
- Deploy dự án bằng **vercel**.

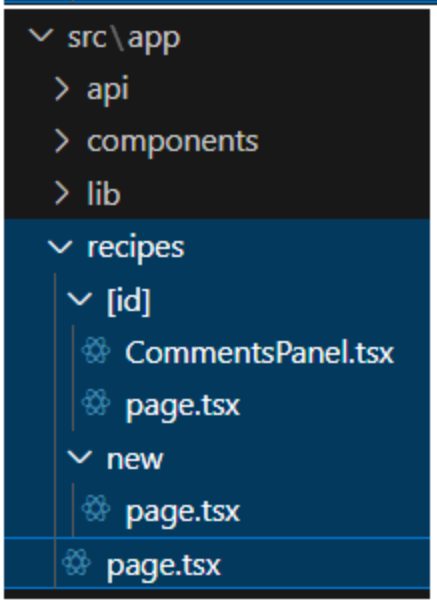
1) Cấu trúc dự án

Tối thiểu có các file/thư mục:




src\app
api
comments
recipes
seed
components
CommentForm.tsx
CommentList.tsx
Header.tsx
LikeButton.tsx
Pagination.tsx
RecipeCard.tsx
RecipeForm.tsx
lib
db.ts
types.ts
recipes
[id]
new
page.tsx
scripts
init.sql
favicon.ico
globals.css
layout.tsx
page.tsx





2) Database schema (PostgreSQL) (15 điểm)

Tạo 3 bảng **recipes**, **comments**, **likes** (tên cột & kiểu gợi ý):
src\app\scripts\init.sql

recipes

```sql
-- recipes
CREATE TABLE
    IF NOT EXISTS recipes (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid (),
        title TEXT NOT NULL,
        description TEXT NOT NULL,
        ingredients JSONB NOT NULL, -- array of strings
        steps JSONB NOT NULL, -- array of strings
        image_url TEXT,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW ()
    );
```

comments

```sql
-- comments
CREATE TABLE
    IF NOT EXISTS comments (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid (),
        recipe_id UUID NOT NULL REFERENCES recipes (id) ON DELETE CASCADE,
        content TEXT NOT NULL,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW ()
    );
```

likes

```sql
CREATE TABLE
    IF NOT EXISTS likes (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid (),
        recipe_id UUID NOT NULL REFERENCES recipes (id) ON DELETE CASCADE,
        fingerprint TEXT NOT NULL,
        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW (),
        CONSTRAINT uniq_recipe_fingerprint UNIQUE (recipe_id, fingerprint)
    );
```

```sql
-- indexes
CREATE INDEX IF NOT EXISTS idx_recipes_created_at ON recipes (created_at DESC);

CREATE INDEX IF NOT EXISTS idx_comments_recipe_id_created_at ON comments (recipe_id, created_at DESC);

CREATE INDEX IF NOT EXISTS idx_likes_recipe_id ON likes (recipe_id);

CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Khai báo kết nối với postgres

src > app > lib > TS db.ts > ...

```
1  import { Pool, QueryResult, QueryResultRow } from "pg";
2
3  if (!process.env.DATABASE_URL) {
4    throw new Error("DATABASE_URL is not set");
5  }
6
7  export const pool = new Pool({
8    connectionString: process.env.DATABASE_URL,
9    ssl: { rejectUnauthorized: false },
10 });
11
12 export type SqlParam = string | number | boolean | Date | Buffer | null;
13
14 export async function query<T extends QueryResultRow = QueryResultRow>(
15   text: string,
16   params: SqlParam[] = []
17 ): Promise<QueryResult<T>> {
18   const client = await pool.connect();
19   try {
20     const res = await client.query<T>(text, params);
21     return res;
22   } finally {
23     client.release();
24   }
25 }
```

Và Types

src > app > lib > TS types.ts > Comment

```typescript
export type Recipe = {
  id: string;
  title: string;
  description: string;
  ingredients: string[];
  steps: string[];
  image_url?: string | null;
  created_at: string;
};

export type Comment = {
  id: string;
  recipe_id: string;
  content: string;
  created_at: string;
};

```

> Khởi tạo database thông qua Seeder src\app\api\seed\route.ts:

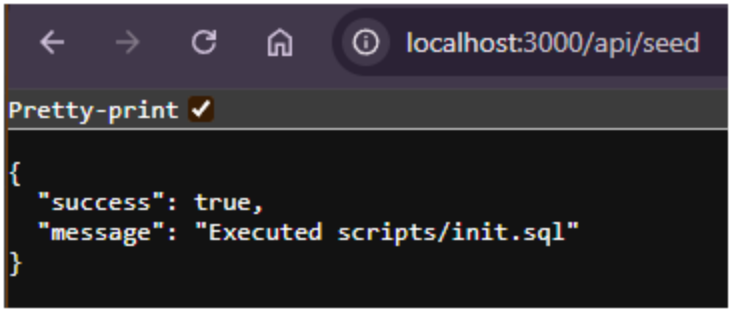
```
import { pool } from "@/app/lib/db";
import { NextResponse } from "next/server";
import fs from "node:fs/promises";
import path from "node:path";

export async function GET(req: Request) {
  const client = await pool.connect();
  try {
    const sqlPath = path.join(process.cwd(), "src/app/scripts", "init.sql");
    const sqlRaw = await fs.readFile(sqlPath, "utf8");
    const sql = sqlRaw.trim();

    await client.query(sql);

    return NextResponse.json({
      success: true,
      message: "Executed scripts/init.sql",
    });
  } catch (err: unknown) {
    const message = err instanceof Error ? err.message : String(err);
    console.error("Seed error:", message, err);
    return NextResponse.json({ error: message }, { status: 500 });
  } finally {
    client.release();
  }
}
```

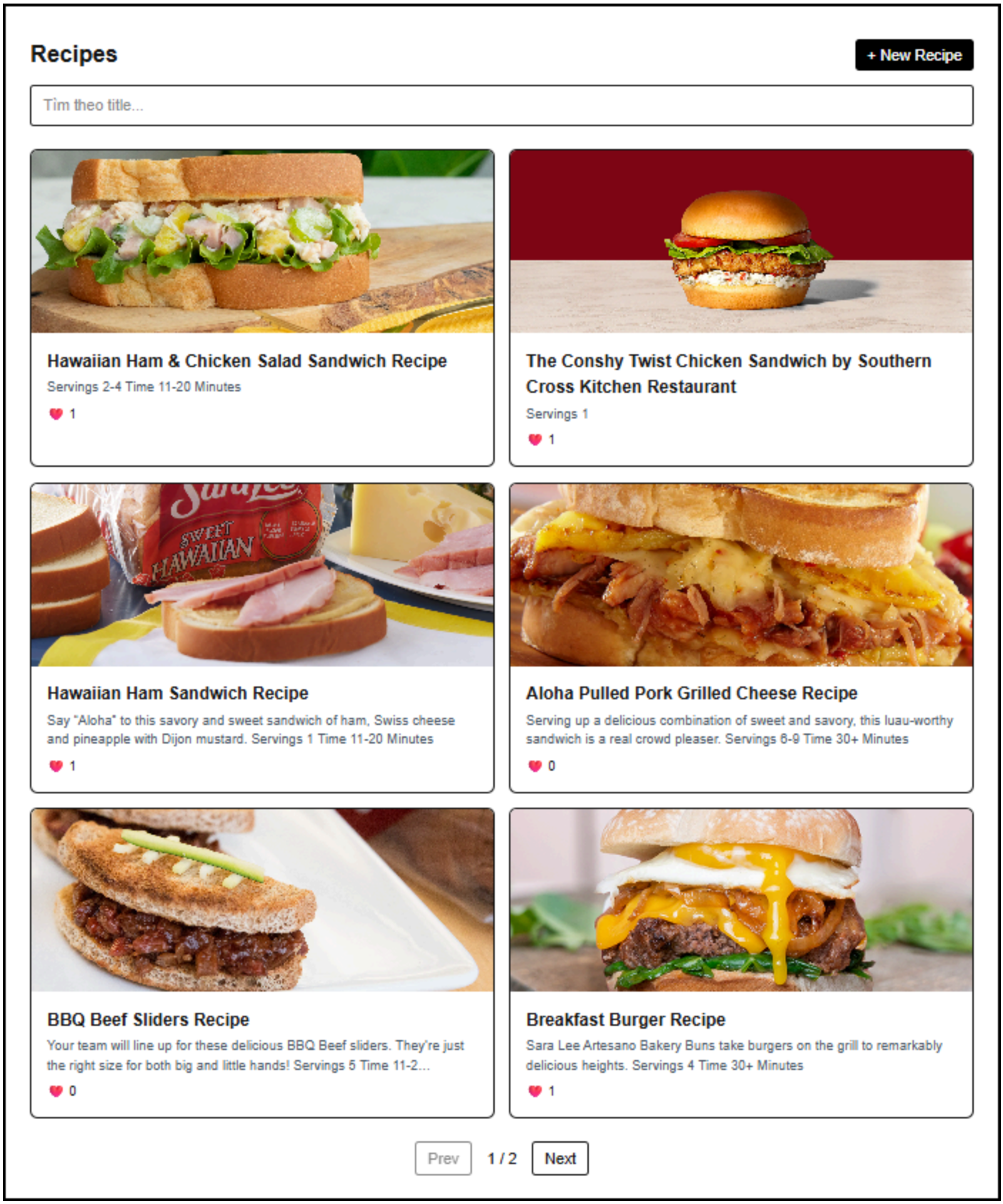
Mở browser và truy cập vào



3) Xây dựng Pages

3.1 `/recipes` – List + Search + Pagination

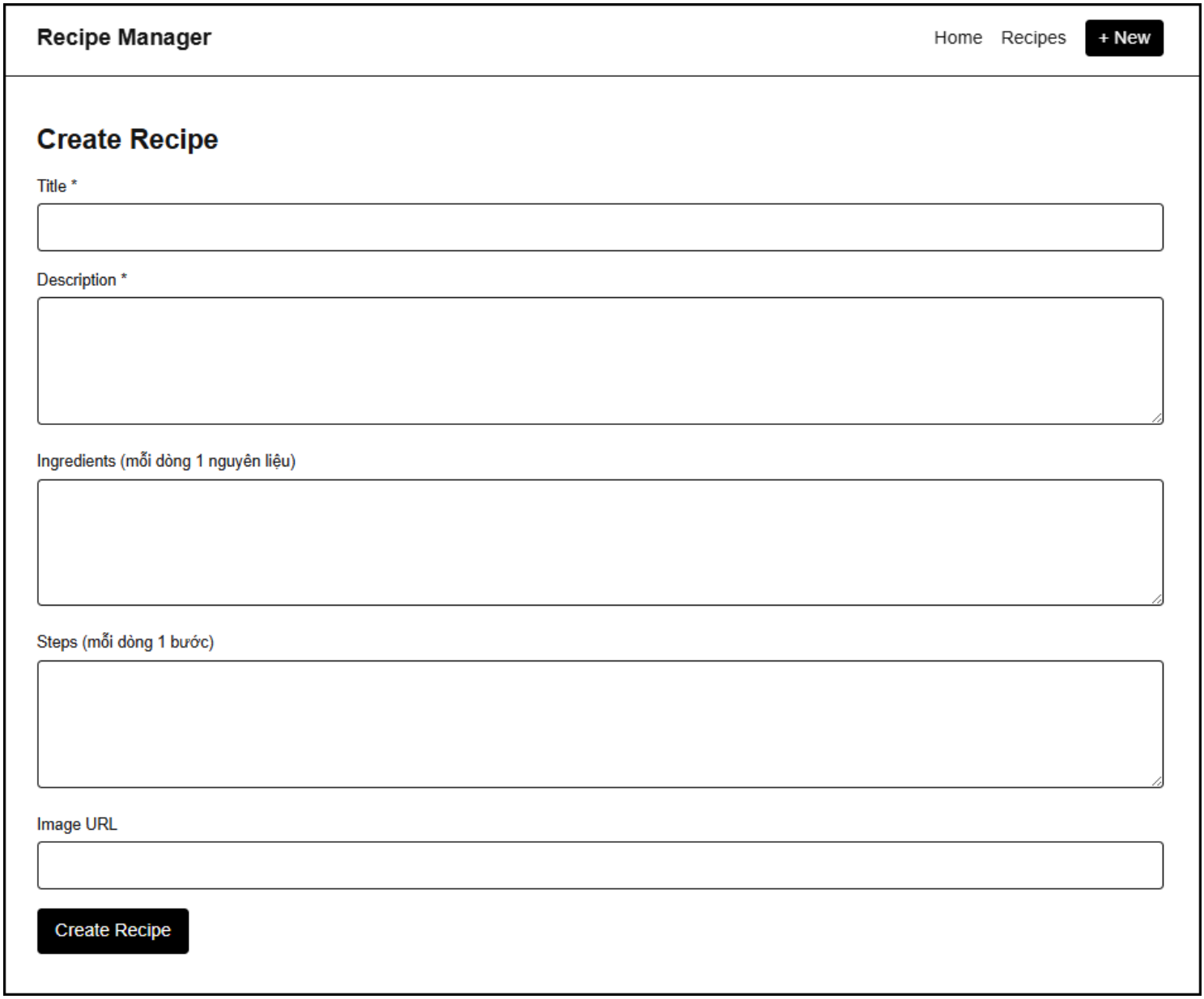
- Server Component fetch data từ API (`/api/recipes`) hoặc trực tiếp từ DB.
- **Search** theo `title` (qua `useSearchParams` , query `?q=...`).
- **Pagination** (VD: `?page=1&limit=6`), hiển thị tối thiểu **6 recipes/trang**.
- Mỗi item hiển thị: `image_url` , `title` , `short description` (cắt ngắn), **like count**.
- Click item → điều hướng `/recipes/[id]` .
- Có nút "+ New Recipe" → `/recipes/new` .



3.2 `/recipes/new` – Create

- Client Component **RecipeForm**:

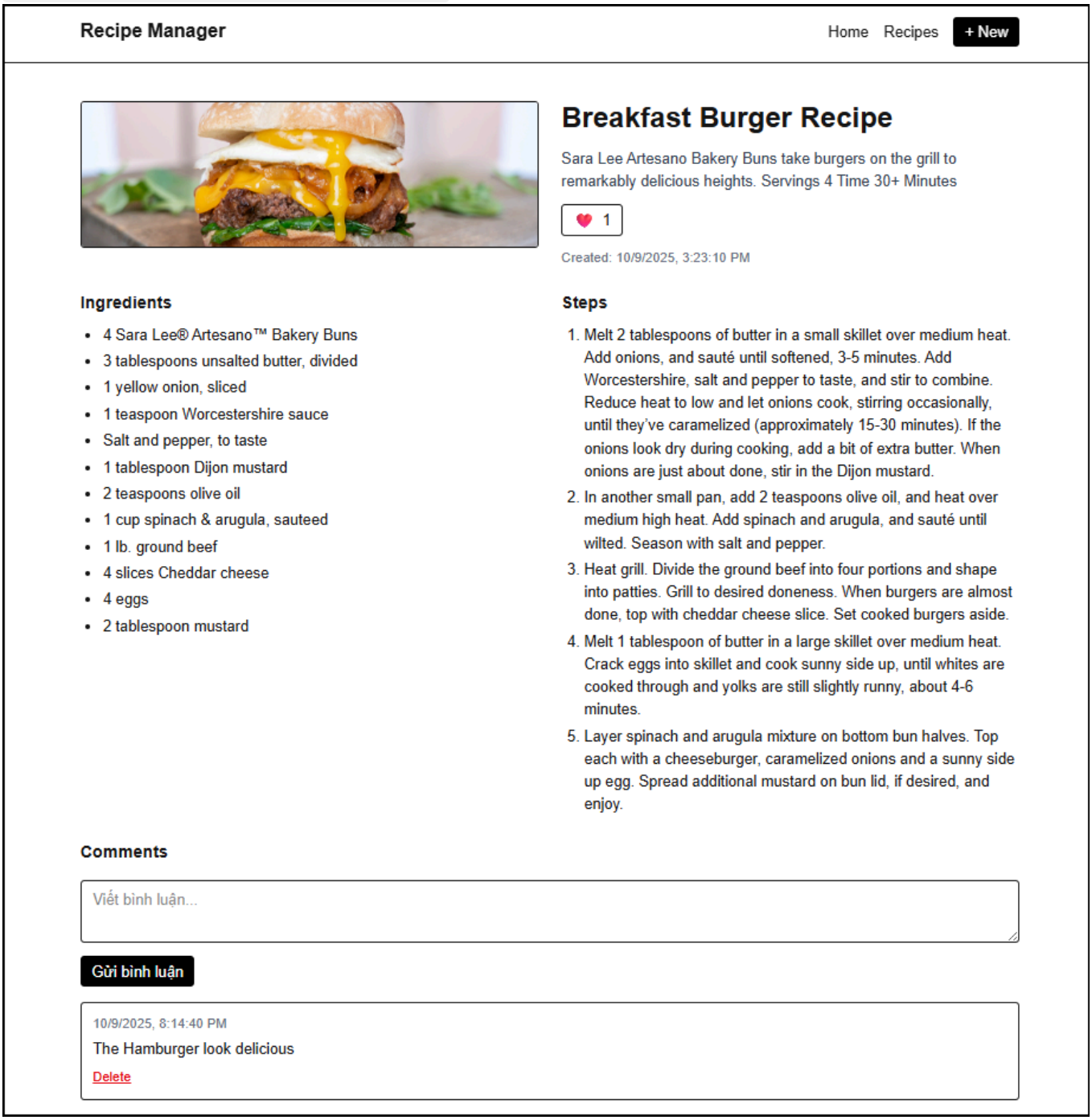
- Input: `title` (required), `description` (required), `ingredients` (textarea, mỗi dòng 1 nguyên liệu), `steps` (textarea), `image_url` (optional).
- Submit → `POST /api/recipes` → redirect về `/recipes` (hoặc `/recipes/[id]`).
- Validate tối thiểu required fields ở client.



3.3 `/recipes/[id]` – Detail + Comments + Like

- Server-side fetch recipe detail + **total likes** + **comments**.
- **LikeButton** (Client Component):
  - Nhấn **Like** → `POST /api/recipes/[id]/like`.
  - Cập nhật đếm likes **không reload trang**.
- **CommentForm** (Client):
  - Nhập `content` → `POST /api/recipes/[id]/comments` → **không reload trang**.
- **CommentList**:

- Hiển thị comments theo `created_at DESC`.
- Mỗi comment có nút Delete (không cần phân quyền người tạo), gọi `DELETE /api/comments/[commentId]`.



4) API Routes:

Chúng ta có các routes cơ bản như sau:

```
route.ts
TS route.ts src\app\api\seed                    recently opened
TS route.ts src\app\api\recipes
TS route.ts src\app\api\recipes\[id]
TS route.ts src\app\api\recipes\[id]\like
TS route.ts src\app\api\comments\[commentId]
TS route.ts src\app\api\recipes\[id]\comments
```

`/api/recipes`

- **GET**: `?q=...&page=1&limit=6` → `{ data: Recipe[], total: number, page: number, limit: number }`
- **POST**: body `{ title, description, ingredients: string[], steps: string[], image_url? }` → `{ data: Recipe }`

src > app > api > recipes > TS route.ts > ...

```
1   import { query, SqlParam } from "@/app/lib/db";
2   import { NextRequest, NextResponse } from "next/server";
3
4   export const dynamic = "force-dynamic";
5
6   // ---- Types ----
7   type CountRow = { count: string };
8
9 > type RecipeRow = {···
17  };
18
19  type RecipeRowWithLikes = RecipeRow & { likes: number | string | null };
20
21 > type CreateRecipePayload = {···
27  };
28
29  // ---- GET /api/recipes ----
30 > export async function GET(req: NextRequest) {···
89  }
90
91  // ---- POST /api/recipes ----
92 > export async function POST(req: NextRequest) {···
151 }
152
```

```
// ---- GET /api/recipes ----
export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url);

  const q = (searchParams.get("q") || "").trim();

  const page = Math.max(1, Number(searchParams.get("page") || "1"));
  const limit = Math.max(
    1,
    Math.min(48, Number(searchParams.get("limit") || "6"))
  );
  const offset = (page - 1) * limit;

  const where: string[] = [];
  const values: SqlParam[] = [];

  if (q) {
    values.push(`%${q}%`);
    where.push(`title ILIKE $${values.length}`);
  }
  const whereSql = where.length ? `WHERE ${where.join(" AND ")}` : "";

  const totalRes = await query<CountRow>(
    `SELECT COUNT(*)::text AS count FROM recipes ${whereSql}`,
    values
  );
  const total = Number(totalRes.rows[0]?.count ?? 0);

  const listRes = await query<RecipeRowWithLikes>(
    `
    SELECT r.*, COALESCE(l.cnt, 0) AS likes
    FROM recipes r
    LEFT JOIN (
      SELECT recipe_id, COUNT(*)::int AS cnt
      FROM likes
      GROUP BY recipe_id
    ) l ON l.recipe_id = r.id
    ${whereSql}
    ORDER BY r.created_at DESC
    LIMIT $${values.length + 1}
    OFFSET $${values.length + 2}
    `,
    [...values, limit, offset]
  );

  return NextResponse.json({
    data: listRes.rows.map((r) => ({
      id: r.id,
      title: r.title,
      description: r.description,
      ingredients: r.ingredients,
      steps: r.steps,
      image_url: r.image_url,
      created_at: r.created_at,
      likes: Number(r.likes ?? 0),
    })),
    total,
    page,
    limit,
  });
}
```

```python
// ---- POST /api/recipes ----
export async function POST(req: NextRequest) {
  try {
    const body = (await req.json()) as CreateRecipePayload;

    const { title, description } = body;
    let { ingredients, steps } = body;
    const image_url = body.image_url ?? null; // const (fixes prefer-const)

    if (!title || !description) {
      return NextResponse.json(
        { error: "title & description are required" },
        { status: 400 }
      );
    }

    // Normalize ingredients
    if (!Array.isArray(ingredients)) {
      if (typeof ingredients === "string") {
        ingredients = ingredients
          .split("\n")
          .map((s) => s.trim())
          .filter(Boolean);
      } else {
        ingredients = [];
      }
    }

    // Normalize steps
    if (!Array.isArray(steps)) {
      if (typeof steps === "string") {
        steps = steps
          .split("\n")
          .map((s) => s.trim())
          .filter(Boolean);
      } else {
        steps = [];
      }
    }

    const ins = await query<RecipeRow>(
      `
      INSERT INTO recipes (title, description, ingredients, steps, image_url)
      VALUES ($1, $2, $3::jsonb, $4::jsonb, $5)
      RETURNING *
      `,
      [
        title,
        description,
        JSON.stringify(ingredients),
        JSON.stringify(steps),
        image_url,
      ]
    );

    return NextResponse.json({ data: ins.rows[0] }, { status: 201 });
  } catch (e: unknown) {
    const message = e instanceof Error ? e.message : "unknown error";
    return NextResponse.json({ error: message }, { status: 500 });
  }
}
```

`/api/recipes/[id]`

- **GET**: → `{ data: Recipe, likes: number }`
- **PUT**: body `{ title?, description?, ingredients?, steps?, image_url? }` → `{ data: Recipe }`

- **DELETE:** → `{ success: true }`

src > app > api > recipes > [id] > TS route.ts > ...

```
1   import { query, SqlParam } from "@/app/lib/db";
2   import { NextRequest, NextResponse } from "next/server";
3
4   type Params = { id: string };
5
6   type RecipeRow = {
7     id: string;
8     title: string;
9     description: string | null;
10    ingredients: unknown; // JSONB
11    steps: unknown; // JSONB
12    image_url: string | null;
13    created_at: string; // or Date if you parse it
14  };
15
16  type UpdateRecipePayload = {
17    title?: string;
18    description?: string;
19    ingredients?: string | string[];
20    steps?: string | string[];
21    image_url?: string | null;
22  };
23
24 > export async function GET(···
44  }
45
46 > export async function PUT(···
100 }
101
102 > export async function DELETE(···
109 }
110
```

```
export async function GET(
  _req: NextRequest,
  context: { params: Promise<Params> }
) {
  const { id } = await context.params;

  const rec = await query<RecipeRow>(`SELECT * FROM recipes WHERE id = $1`, [
    id,
  ]);
  if (rec.rowCount === 0) {
    return NextResponse.json({ error: "Not found" }, { status: 404 });
  }

  const likesRes = await query<{ count: string }>(
    `SELECT COUNT(*)::text AS count FROM likes WHERE recipe_id = $1`,
    [id]
  );
  const likes = Number(likesRes.rows[0]?.count ?? 0);

  return NextResponse.json({ data: rec.rows[0], likes });
}
```

```
export async function PUT(
  req: NextRequest,
  context: { params: Promise<Params> }
) {
  const { id } = await context.params;
  const body = (await req.json()) as UpdateRecipePayload;

  const fields: string[] = [];
  const values: SqlParam[] = [];
  let i = 1;

  const setField = (k: string, v: unknown, json = false) => {
    if (v !== undefined) {
      fields.push(`${k} = $${i++}`);
      values.push(json ? JSON.stringify(v) : (v as SqlParam));
    }
  };

  setField("title", body.title);
  setField("description", body.description);

  let ingredients = body.ingredients;
  if (!Array.isArray(ingredients) && typeof ingredients === "string") {
    ingredients = ingredients
      .split("\n")
      .map((s) => s.trim())
      .filter(Boolean);
  }
  setField("ingredients", ingredients, true);

  let steps = body.steps;
  if (!Array.isArray(steps) && typeof steps === "string") {
    steps = steps
      .split("\n")
      .map((s) => s.trim())
      .filter(Boolean);
  }
  setField("steps", steps, true);

  setField("image_url", body.image_url ?? null);

  if (fields.length === 0) {
    return NextResponse.json({ error: "No fields to update" }, { status: 400 });
  }

  const sql = `UPDATE recipes SET ${fields.join(
    ", "
  )} WHERE id = $${i} RETURNING *`;
  const res = await query<RecipeRow>(sql, [...values, id]);

  if (res.rowCount === 0) {
    return NextResponse.json({ error: "Not found" }, { status: 404 });
  }
  return NextResponse.json({ data: res.rows[0] });
}
```

```
export async function DELETE(
  _req: NextRequest,
  context: { params: Promise<Params> }
) {
  const { id } = await context.params;
  await query(`DELETE FROM recipes WHERE id = $1`, [id]);
  return NextResponse.json({ success: true });
}
```

`/api/recipes/[id]/like`

- **POST**: body `{ fingerprint: string }`

src > app > api > recipes > [id] > like > TS route.ts > POST

```
1   import { query } from "@/app/lib/db";
2   import { NextRequest, NextResponse } from "next/server";
3
4   type Params = { id: string };
5
6   export async function POST(
7     req: NextRequest,
8     context: { params: Promise<Params> }
9   ) {
10    const { id: recipeId } = await context.params;
11
12    const  const fingerprint: any  req.json();
13    if (!fingerprint) {
14      return NextResponse.json(
15        { error: "fingerprint required" },
16        { status: 400 }
17      );
18    }
19
20    try {
21      await query(
22        `INSERT INTO likes (recipe_id, fingerprint)
23         VALUES ($1, $2)
24         ON CONFLICT (recipe_id, fingerprint) DO NOTHING`,
25        [recipeId, fingerprint]
26      );
27
28      const check = await query(
29        `SELECT 1 FROM likes WHERE recipe_id = $1 AND fingerprint = $2`,
30        [recipeId, fingerprint]
31      );
32
33      let liked: boolean;
34      if (check.rowCount && check.rowCount > 0) {
35        liked = true;
36      } else {
37        await query(
38          `DELETE FROM likes WHERE recipe_id = $1 AND fingerprint = $2`,
39          [recipeId, fingerprint]
40        );
41        liked = false;
42      }
43
44      const countRes = await query<{ count: string }>(
45        `SELECT COUNT(*)::text AS count FROM likes WHERE recipe_id = $1`,
46        [recipeId]
47      );
48
49      return NextResponse.json({
50        liked,
51        likes: Number(countRes.rows[0]?.count ?? 0),
52      });
53    } catch (e: unknown) {
54      const message =
55        e instanceof Error ? e.message : "unknown error during like toggle";
56      return NextResponse.json({ error: message }, { status: 500 });
57    }
58  }
```

/api/recipes/[id]/comments

- **GET:** → `{ data: Comment[] }`
- **POST:** body `{ content: string }` → `{ data: Comment }`

src > app > api > recipes > [id] > comments > TS route.ts > ...

```ts
import { query } from "@/app/lib/db";
import { NextRequest, NextResponse } from "next/server";

type Params = { id: string };

export async function GET(
  _req: NextRequest,
  context: { params: Promise<Params> }
) {
  const { id: recipeId } = await context.params;

  const res = await query(
    `SELECT * FROM comments WHERE recipe_id = $1 ORDER BY created_at DESC`,
    [recipeId]
  );

  return NextResponse.json({ data: res.rows });
}

export async function POST(
  req: NextRequest,
  context: { params: Promise<Params> }
) {
  const { id: recipeId } = await context.params;

  const { content } = await req.json();
  if (!content || !content.trim()) {
    return NextResponse.json({ error: "content required" }, { status: 400 });
  }

  const ins = await query(
    `INSERT INTO comments (recipe_id, content) VALUES ($1, $2) RETURNING *`,
    [recipeId, content.trim()]
  );

  return NextResponse.json({ data: ins.rows[0] }, { status: 201 });
}

```

/api/comments/[commentId]

- **DELETE:** → `{ success: true }`

> src\app\api\comments[commentId]\route.ts

```
import { query } from "@/app/lib/db";
import { NextResponse, type NextRequest } from "next/server";

export async function DELETE(
  _req: NextRequest,
  { params }: { params: Promise<{ commentId: string }> }
) {
  const { commentId } = await params;

  await query(`DELETE FROM comments WHERE id=$1`, [commentId]);
  return NextResponse.json({ success: true });
}
```

> Lưu ý: **Không cần user**;

5) UI/UX

- `Header` cố định trên cùng: “Recipe Manager” + nav: Home | Recipes | New
- Responsive đơn giản (grid list 1 cột mobile, ≥2 cột desktop).

6) Env & Deploy (bắt buộc)

Hàm tiện ích khi deploy

```
function getBaseUrl() {
  const fromEnv =
    process.env.NEXT_PUBLIC_BASE_URL || process.env.VERCEL_URL || "";

  if (fromEnv) {
    return fromEnv.startsWith("http") ? fromEnv : `https://${fromEnv}`;
  }
  return "http://localhost:3000";
}
```

- Dùng **biến môi trường** (đặt vào `.env`):

```
# Neon
DATABASE_URL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-pooler.ap
DATABASE_URL_UNPOOLED=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3.
PGHOST=ep-young-fog-a1vkvmy3-pooler.ap-southeast-1.aws.neon.tech
PGHOST_UNPOOLED=ep-young-fog-a1vkvmy3.ap-southeast-1.aws.neon.tech
PGUSER=neondb_owner
PGDATABASE=neondb
PGPASSWORD=npg_mFa5LASgPbW6
POSTGRES_URL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-pooler.ap
POSTGRES_URL_NON_POOLING=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvm
POSTGRES_USER=neondb_owner
POSTGRES_HOST=ep-young-fog-a1vkvmy3-pooler.ap-southeast-1.aws.neon.tech
POSTGRES_PASSWORD=npg_mFa5LASgPbW6
POSTGRES_DATABASE=neondb
POSTGRES_URL_NO_SSL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-po
POSTGRES_PRISMA_URL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3.ap
```

- Deploy **Vercel**: cấu hình `Build Command`, `Output Directory` mặc định Next.js.
- Đảm bảo app chạy được trên domain Vercel, có CRUD + comments + like hoạt động.
- Deploy thì sử dụng biến môi trường bên dưới:

```
# Khai báo khi deploy thay 'recipe-mg' thành tên domain của mình!!!
NEXT_PUBLIC_BASE_URL=https://recipe-mg.vercel.app
VERCEL_URL=recipe-mg.vercel.app

# Recommended for most uses
DATABASE_URL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-pooler.ap

# For uses requiring a connection without pgbouncer
DATABASE_URL_UNPOOLED=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3.

# Parameters for constructing your own connection string
PGHOST=ep-young-fog-a1vkvmy3-pooler.ap-southeast-1.aws.neon.tech
PGHOST_UNPOOLED=ep-young-fog-a1vkvmy3.ap-southeast-1.aws.neon.tech
PGUSER=neondb_owner
PGDATABASE=neondb
PGPASSWORD=npg_mFa5LASgPbW6

# Parameters for Vercel Postgres Templates
POSTGRES_URL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-pooler.ap
POSTGRES_URL_NON_POOLING=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvm
POSTGRES_USER=neondb_owner
POSTGRES_HOST=ep-young-fog-a1vkvmy3-pooler.ap-southeast-1.aws.neon.tech
POSTGRES_PASSWORD=npg_mFa5LASgPbW6
POSTGRES_DATABASE=recipe-mg
POSTGRES_URL_NO_SSL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-po
POSTGRES_PRISMA_URL=postgresql://neondb_owner:npg_mFa5LASgPbW6@ep-young-fog-a1vkvmy3-po

# Neon Auth environment variables for Next.js
NEXT_PUBLIC_STACK_PROJECT_ID=****************************
NEXT_PUBLIC_STACK_PUBLISHABLE_CLIENT_KEY=****************************************
STACK_SECRET_SERVER_KEY=**********************
```